CS300-ANALYSIS & DESIGN
JON SEGUIN
10-11-2026

MENU PSUEDOCODE

START

Create an empty binary search tree.

Display the menu:

1. Load Data Structure

2. Print Course List

3. Print Course Information

9. Exit

Ask the user to enter a menu option.

WHILE the user does not select 9

IF the user selects 1

    Open the course data file.

    Read each course from the file.

    Create a course object for each line.

    Store each course in the binary search tree.

    Close the file.

    Display "Courses loaded successfully."

ELSE IF the user selects 2

CS300-ANALYSIS & DESIGN
JON SEGUIN
10-11-2026

```
    IF the tree is empty
        Display "Please load the data first."
    ELSE
        Traverse the tree in order.
        Print the course number and course title
        for each course.
    END IF

ELSE IF the user selects 3
    IF the tree is empty
        Display "Please load the data first."
    ELSE
        Ask the user to enter a course number.
        Search the binary search tree for the course.

        IF the course is found
            Print the course number.
            Print the course title.
            Print the prerequisites.
        ELSE
            Display "Course not found."
        END IF
    END IF
```

ELSE

Display "Invalid menu option."

END IF

Display the menu again.

Ask the user to enter a menu option.

END WHILE

IF the user selects 9

Display "Goodbye."

END IF

END

PSUEDOCODE: PRINT IN ORDER

START

IF the binary search tree is empty

Display "Please load the course data first."

ELSE

Set the current node to the root of the tree.

Call the in-order traversal function.

END IF

FUNCTION InOrderTraversal(current node)

IF the current node is not empty

Call InOrderTraversal on the left child.

Display the course number and course title.

Call InOrderTraversal on the right child.

END IF

END FUNCTION

END

EVALUATION

The worst-case running time for reading the file, validating prerequisites, creating course objects, and inserting them into an unbalanced binary search tree is $O(n^2)$. This is because prerequisite checks may require searching through all course numbers, and tree insertions can take up to O(n)

time each. The memory complexity is O(n) because the program stores course objects, tree nodes, and course numbers for validation.

A vector is easy to use and allows courses to be stored in a simple list. It is efficient for adding courses to the end, but searching for a specific course can take O(n) time. The vector must also be sorted before printing courses in alphanumeric order.

A hash table allows courses to be searched quickly by course number, with an average search time of O(1). It is useful for finding courses and checking prerequisites. However, collisions can slow down searches, and the courses are not automatically stored in sorted order.

A binary search tree stores courses by course number and can print them in alphanumeric order using in-order traversal. Searching and inserting can take O(log n) time in a balanced tree, but an unbalanced tree can require O(n) time. It also requires additional memory for tree nodes and links.

Overall, the binary search tree is a good choice for ABCU because advisors need to search for individual courses and print the complete course list in alphanumeric order. A hash table would be better for quick lookups, while a vector would be simpler to implement but less efficient for searching and sorting large amounts of course data.

Hence, I recommend using a binary search tree for ABCU's advising program. A balanced binary search tree can search for courses and insert new courses in O(log n) time. It can also print courses in alphanumeric order using in-order traversal without requiring a separate sorting algorithm. Although an unbalanced tree can have a worst-case runtime of O(n), it meets the advisor's requirements for searching and displaying courses efficiently when balanced. A

hash table offers faster average lookups at O(1), but it does not automatically sort courses. A vector is simpler to use but requires additional time to search and sort the course list. Overall, a balanced binary search tree provides the best combination of efficient searching and ordered course listings for ABCU.